PROGRAMACIÓN AVANZADA

WEB SERVICES



Ing. Matias Macrino

WEB SERVICES

¿Qué es un Web Services?

Un sistema de software, que soporta la interacción de máquina a máquina de forma interoperable.

También definidos como una vía de intercomunicación e interoperabilidad entre máquinas que se encuentran conectadas a una red.

Su principal ventaja es que no tienen una interfaz gráfica (UI) de comunicación.

WEB SERVICES

¿Qué es un Web Services?

Pueden escribirse en diversas tecnologías (.NET, Java, Pearl, etc)

Pueden ser públicos o privados (Internet o Intranet).

Comparten lógica de negocio, datos y los procesos. Conectando programas con programas, por lo tanto son programas que no interactúan directamente con el cliente.

WEB SERVICES

Tipos de Web Services - SOAP:

- Fueron los primeros en aparecer con la revolución de internet.

- Es un protocolo que define cómo deben de realizarse las comunicaciones entre las máquinas.

- Utiliza XML como lenguaje para el intercambio de datos, con una estructura compleja siendo capaz de albergar todo tipo de datos.

- Son web service tipados.

WEB SERVICES

Tipos de Web Services - SOAP:

Compuesto por XML, SOAP, WSDL y UDDI.

XML: Permite a los desarrolladores crear sus propios tag, permitiendo habilitar definiciones, transmisiones, validaciones e interpretación entre datos de las aplicaciones.

SOAP: Simple Object Acces Protocol, protocolo de mensajería construido en XML, utilizado para codificar información de los requerimientos de los WS. SOAP es independiente de los sistemas operativos y pueden ser transportados protocolos de internet (SMTP, HTTP, etc).

WEB SERVICES

Tipos de Web Services - SOAP:

Compuesto por XML, SOAP, WSDL y UDDI.

WSDL: Web Services Description Language, lenguaje especificado en XML que se ocupa de definir los WS como colecciones de punto de comunicaciones capaces de intercambiar mensajes. Es un estándar de uso público.

UDDI: Universal Description, Discovery and Integration. Directorio distribuido que opera en la web, permitiendo a entidades exponer sus propios Web Services, para que otros los conozcan y utilicen los Web Services publicados.

WEB SERVICES

Web Services SOAP - Framework:

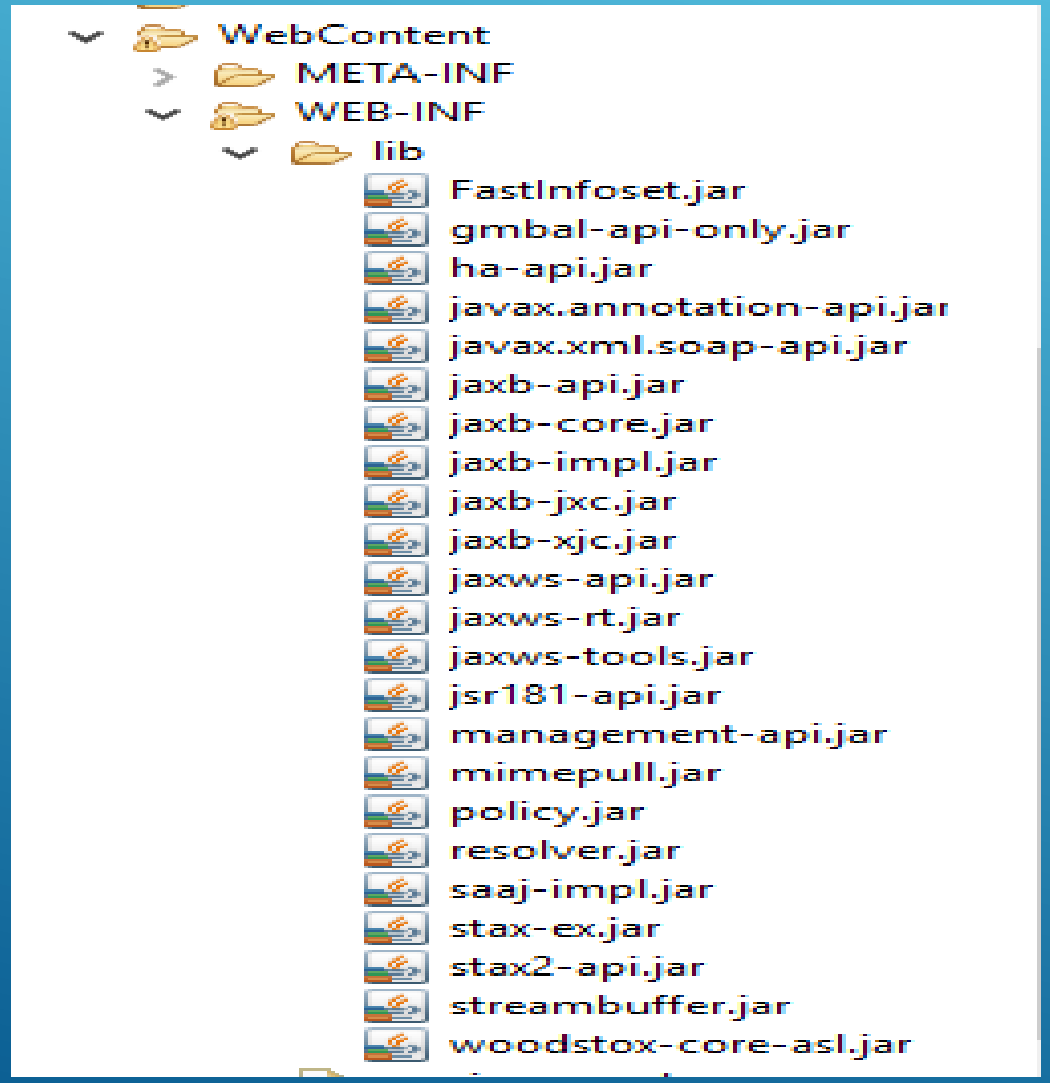
Los dos Framework más populares para el armado de Web Services en Java son ***Axis*** y ***JAX-WS*** ambos de código abierto. El primero es de ***Apache*** y el segundo de ***GlassFish***

Para poder tener un servicio funcional con Axis, además de la codificación del mismo requerimos de varios XML de configuración.

En el caso de JAX-WS, podemos tener el servicio funcional más rápido, ya que toda la configuración que requiere Axis, la vamos a resolver a través de anotaciones.

WEB SERVICES

Web Services SOAP – JAX-WS:



Crear un Dynamic Web Project

Copiar las librerías descargadas dentro de ***lib*** del proyecto.

WEB SERVICES

Web Services SOAP – JAX-WS:

Configuración del Endpoint del Servicio:

- Primero se tiene que crear la ***Interfaz del Endpoint*** del Web Services. Esta interfaz contendrá la definición de los métodos que se incluirán en el servicio.

Para la definición de que será un Web Services utilizamos la anotación ***@WebSerevice*** en la definición de nuestra interfaz.

Para la definición de los métodos que contendrá nuestro Web Servicies, se utiliza la anotación ***@WebMethod***

```
import javax.jws.WebMethod;
import javax.jws.WebService;

@WebService
public interface ServicioPrueba {

    @WebMethod
    public String saludar();

}
```

WEB SERVICES

Web Services SOAP – JAX-WS:

Configuración del Endpoint del Servicio:

- Segundo se tiene que crear la clase que implemente la Interfaz creada en el punto anterior. Esta será nuestra implementación del Endpoint

En este caso también usaremos la anotación ***@WebService*** con la diferencia de la Interfaz, debemos de definir la propiedad ***endpointInterface*** de la anotación en cuestión. En dicha propiedad debemos de colocar el ***Qualified Name de la Intefaz***

```
import javax.jws.WebService;

import interfaces.ServicioPrueba;

@WebService(endpointInterface="interfaces.ServicioPrueba")
public class ServicioPruebaImpl implements ServicioPrueba{

	@Override
	public String saludar() {
		return "Hola Mundo!!!";
	}

}
```

WEB SERVICES

Web Services SOAP – JAX-WS:

Configuración del archivo web.xml

- Se tiene que configurar el ***servlet*** que levantara nuestro ***Web Service*** para que se encuentre activo a las peticiones a través de los ***tag <servlet>*** y agregar el ***listner*** que requiere JAX-WS a través de los ***tag <listener>***

```
<web-app>
    <listener>
        <listener-class>
            com.sun.xml.ws.transport.http.servlet.WSServletContextListener
        </listener-class>
    </listener>
    <servlet>
        <servlet-name>ServicioWeb</servlet-name>
        <servlet-class>
            com.sun.xml.ws.transport.http.servlet.WSServlet
        </servlet-class>
        <load-on-startup>1</load-on-startup>
    </servlet>
    <servlet-mapping>
        <servlet-name>ServicioWeb</servlet-name>
        <url-pattern>/*</url-pattern>
    </servlet-mapping>
</web-app>
```

WEB SERVICES

Web Services SOAP – JAX-WS:

Configuración del archivo sun-jaxws.xml

- Se tiene que crear el archivo ***sun-jaxws.xml*** dentro de ***WEB-INF***, en donde se definirá el ***Endpoint*** del Servicio en donde se declara la clase de la implementación del proyecto. En donde también se definirá la ***URL del servicio***.

```
<endpoints xmlns="http://java.sun.com/xml/ns/jax-ws/ri/runtime"
    version="2.0">
    <endpoint name="ServicioUno"
        implementation="implementaciones.ServicioPruebaImpl"
        url-pattern="/ServicioUno" />
</endpoints>
```

WEB SERVICES

Web Services SOAP – JAX-WS:

Probando el Web Services:

- Se generará el artefacto (archivo *.war) que terminaremos haciendo el despliegue en nuestro web server (Tomcat).

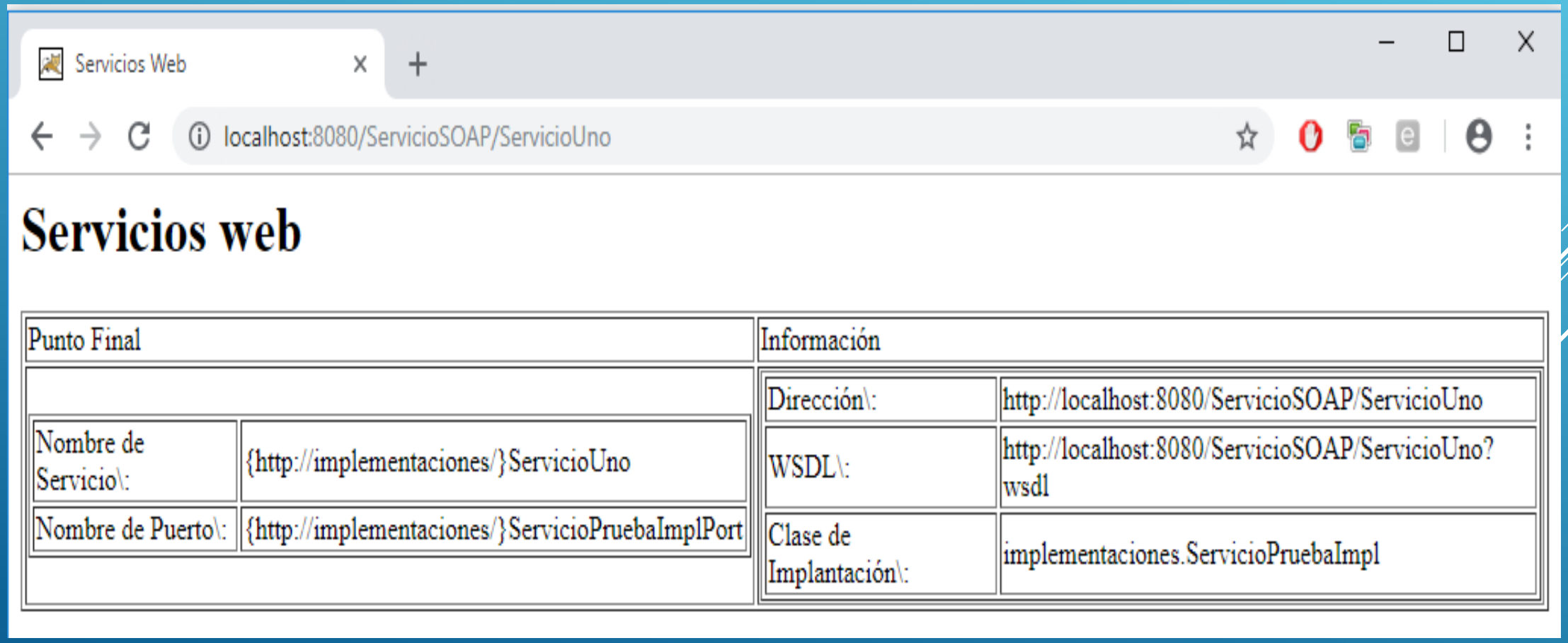
- Accederemos a través del browser para obtener el Endpoint final que expondrá nuestro servicio.

- Si el Web Services se desplegó de manera correcta, obtendremos toda la información de cómo acceder a nuestro contrato expuesto por el servicio (WSDL)

WEB SERVICES

Web Services SOAP – JAX-WS:

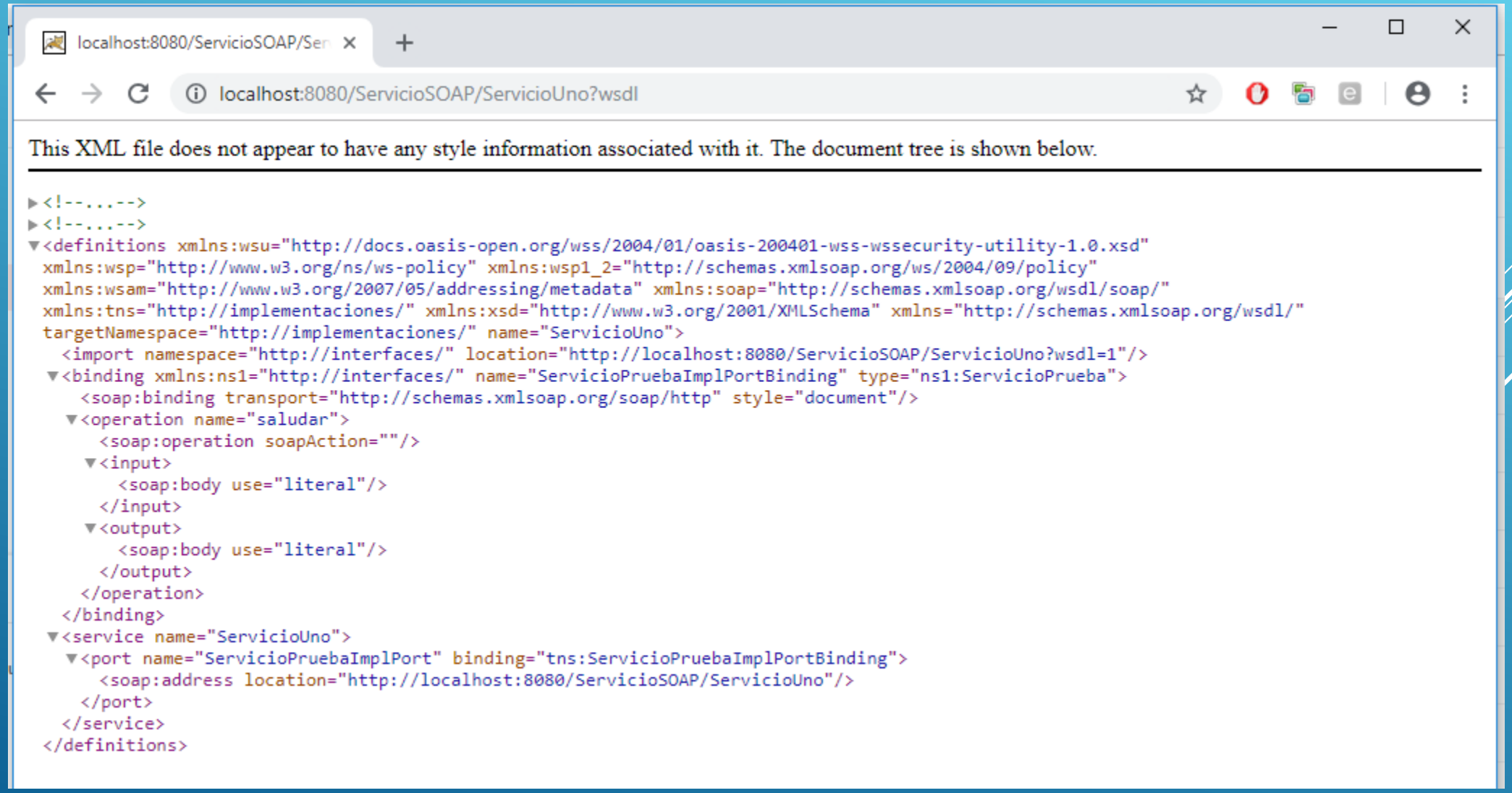
Probando el Web Services:



WEB SERVICES

Web Services SOAP – JAX-WS:

Probando el Web Services WSDL:



WEB SERVICES

Web Services SOAP – JAX-WS:

Probando el Web Services SoapUI:

- Se disponen de varias alternativas para poder probar un servicio web funcional, la mas sencilla y fácil es usando la aplicación de escritorio ***SoapUI***.

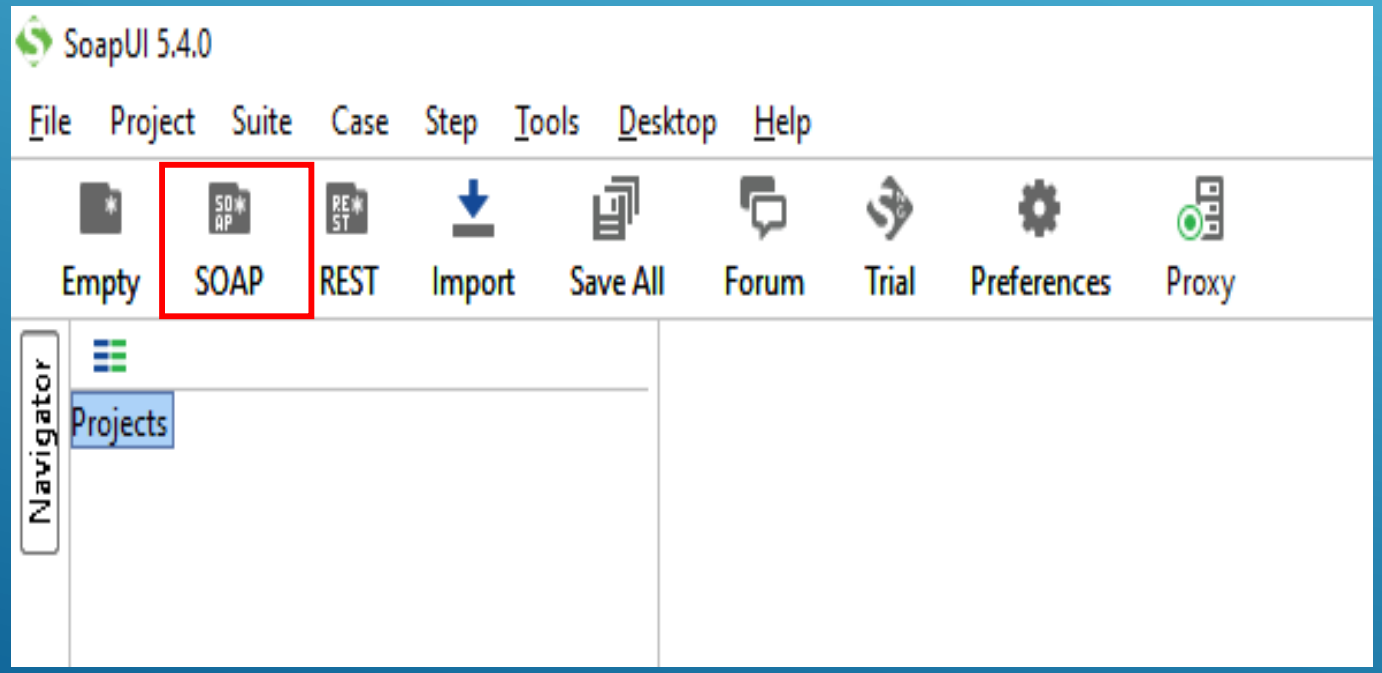
- ***SoapUI*** es un aplicación desarrollada en ***Java*** que nos permite realizar pruebas contra servicios web. Podemos probar diversos tipos de servicios tales como SOAP y REST dentro de otros.

- Actualmente existe dos versiones de ***SoapUI***, una paga y una gratuita open source. Dicho herramienta es desarrollada bajo la compañía SmartBear.

WEB SERVICES

Web Services SOAP – JAX-WS:

Probando el Web Services SoapUI:



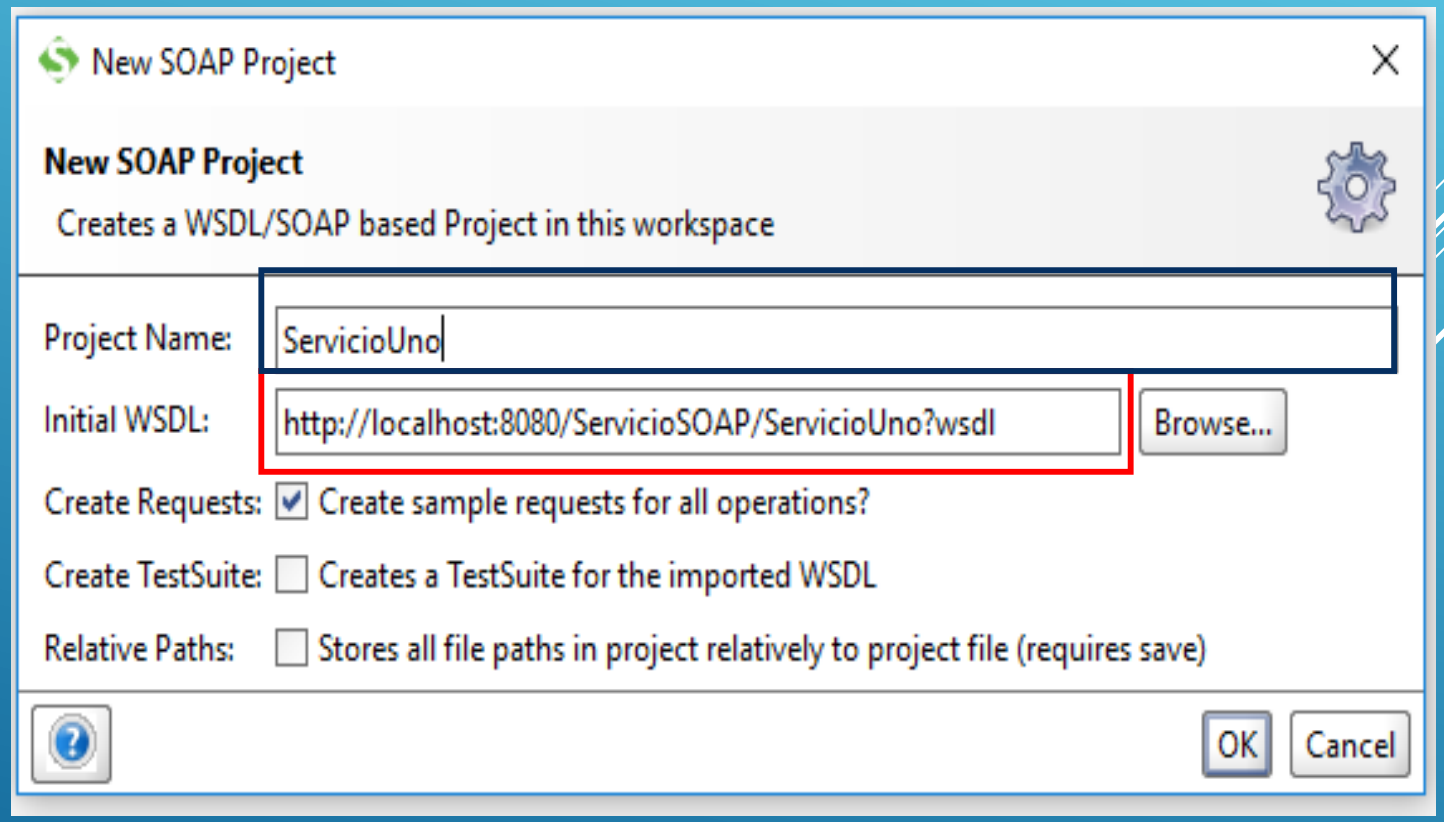
- Iniciamos la aplicación y pulsamos sobre el icono de SAOP.

WEB SERVICES

Web Services SOAP – JAX-WS:

Probando el Web Services SoapUI:

- Ingresamos el WSDL en "***Initial WSDL***" y un nombre a elección en "***Project Name***".



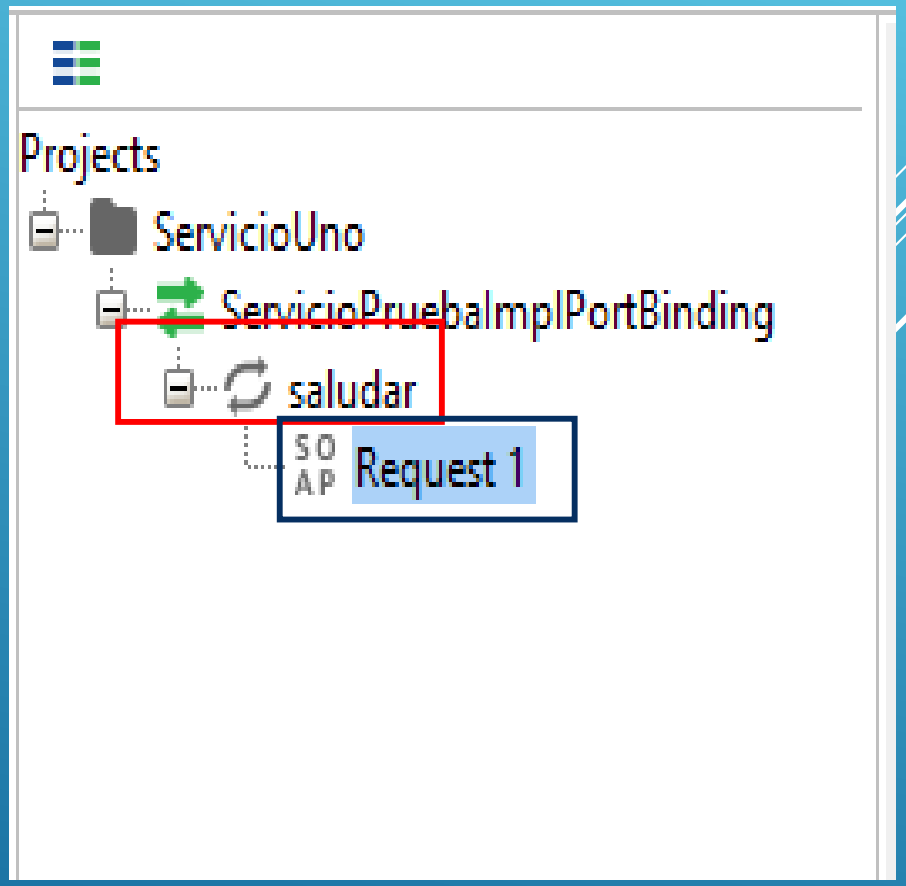
WEB SERVICES

Web Services SOAP – JAX-WS:

Probando el Web Services SoapUI:

Al ingresar los datos necesarios nos armara el proyecto en lo cual veremos la estructura del servicio y los métodos que expone el mismo.

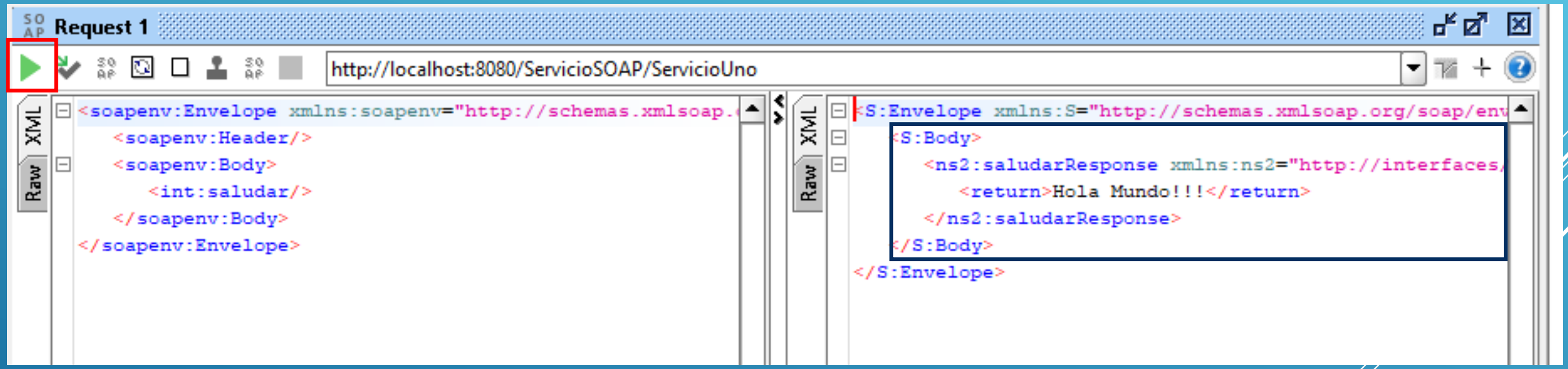
Para consumir un método, debemos de seleccionar el ***Request*** correspondiente al método.



WEB SERVICES

Web Services SOAP – JAX-WS:

Probando el Web Services SoapUI:



En la pantalla de prueba, vamos a tener posibilidad de ver tanto el Request (Izquierda) como el Response (Derecha).

Para accionar el Request solo pulsamos el botón de Play y obtendremos el resultado que nos brinda el servicio.

WEB SERVICES

Web Services SOAP – JAX-WS:

Web Services métodos con argumentos:

```java
@WebService
public interface ServicioPrueba {

    @WebMethod
    public String saludarByNombre(String nombre);
```

```java
@WebService(endpointInterface="interfaces.ServicioPrueba")
public class ServicioPruebaImpl implements ServicioPrueba{

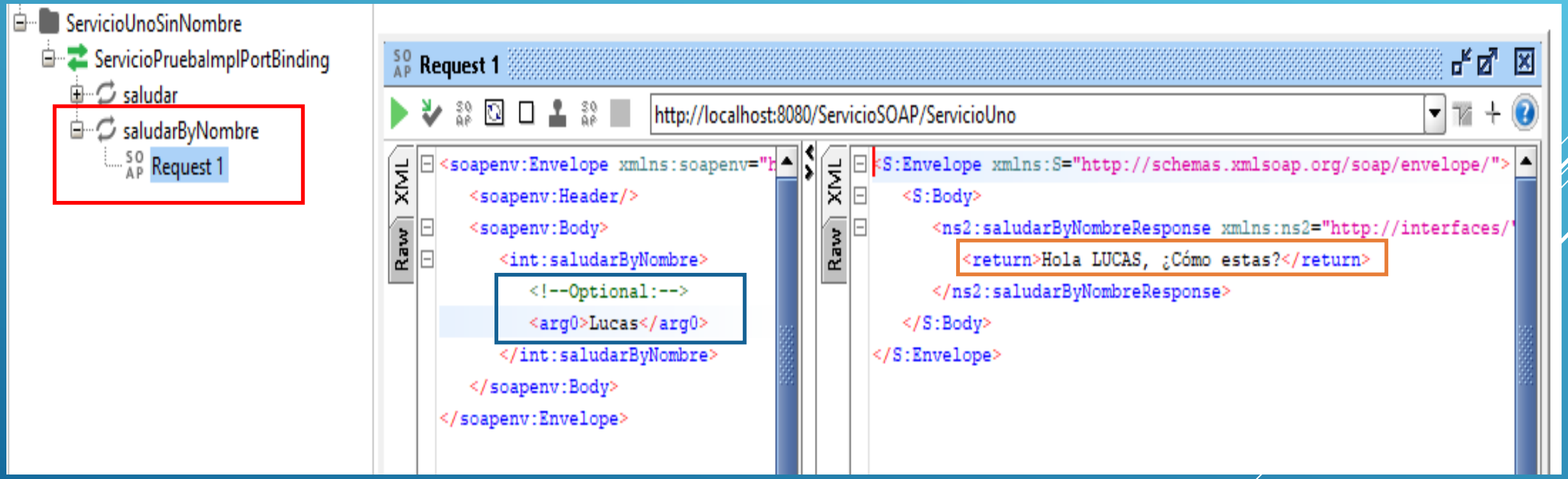
    @Override
    public String saludarByNombre(String nombre) {
        return "Hola " + nombre.toUpperCase() + ", ¿Cómo estas?";
    }
```

Como cualquier programa en Java, agregamos un método nuevo que reciba un parámetro como argumento de entrada. Si el mismo queremos que sea expuesto no hay que olvidar la anotación ***@WebMethod***.

WEB SERVICES

Web Services SOAP – JAX-WS:

Web Services métodos con argumentos - Prueba:



Volvemos a crear el proyecto dentro del SoapUI, en donde encontramos el nuevo Request del método expuesto. Nos encontramos con los tag <arg0> </arg0>

WEB SERVICES

Web Services SOAP – JAX-WS:

Web Services métodos con argumentos - Nombre:

Si agregamos parámetros sea uno o mas, al interpretar el WSDL nos aparecerán siempre con el tag ***<arg0> </arg0> <arg1> </arg1>***, etc. sin conocer el nombre propio del parámetro.
Para solucionar esto, el Framework ***JAX-WS*** nos brinda la anotación ***@WebParam***

```
@WebService
public interface ServicioPrueba {

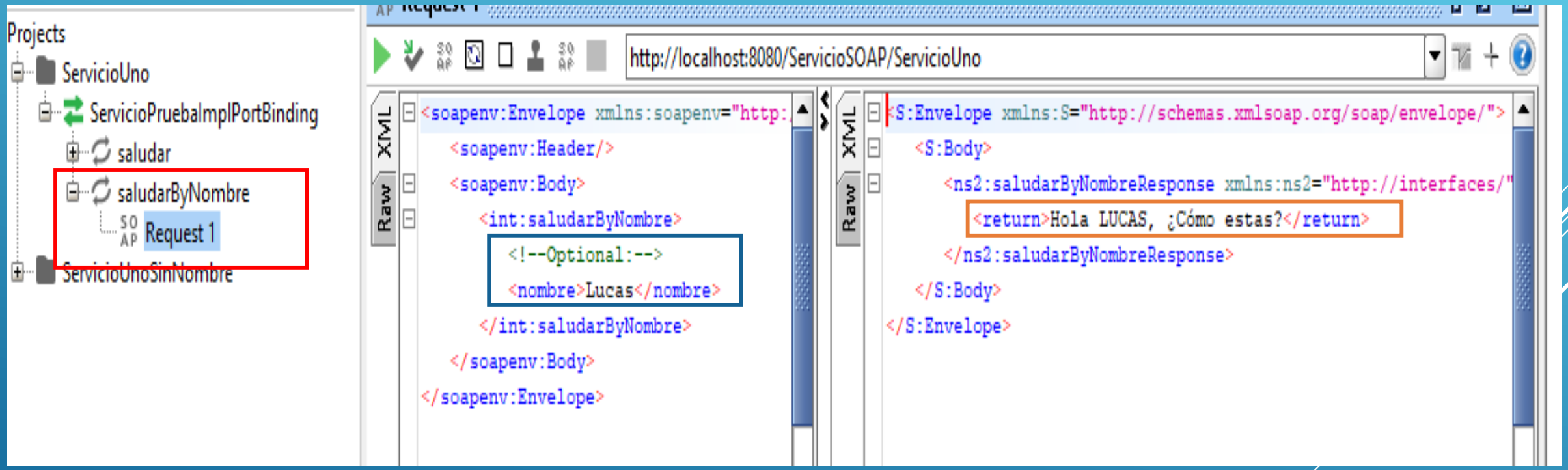
    @WebMethod
    public String saludarByNombre(@WebParam(name="nombre")String nombre);
```

- La anotación ***@WebParam*** tiene un atributo ***name*** el cual será el que luego interpretara el ***WSDL***

WEB SERVICES

Web Services SOAP – JAX-WS:

Web Services métodos con argumentos - Prueba:



Volvemos a crear el proyecto dentro del SoapUI, en donde encontramos el nuevo Request del método expuesto. Nos encontramos con los tag con el nombre del atributo de la anotación ***@WebParam***